



Create customized reports

SQL views are a technique for creating customized reports to show data that cannot be shown with existing models' fields and views. In other words, this technique helps avoid unnecessary creation and calculation of additional fields solely for data analysis purposes.

Create a model

A SQL view is created in a similar manner as a standard model:

```
from odoo import fields, models

class ModuleReport(models.Model):
    _name = 'module.report'
    _description = "Module Report"
    _rec_name = 'module_field'
    _auto = False
```

Where the attributes:

- `_auto = False` indicates that we do not want to store the model in the database
- `_rec_name` indicates which of the model's fields represents a record's name (i.e. the name that will be used in the navigation breadcrumb when opening a record's form view)

and its fields are defined in the same way as a standard model, except every field is marked as `readonly=True`.

Note

Don't forget to add your new model to your security file.

Populate the model

There are 2 ways to populate a SQL view's table:

- override the `BaseModel.init()` method,

you are expected to keep in mind that you are bypassing the ORM (i.e. it is a good idea to read through [Security in Odoo](#) if you haven't already). The columns returned from the `SELECT` will populate the model's fields, so ensure that your column names match your field names, or use alias names that match.

Overriding `BaseModel.init()`

Using `_table_query`

In most cases, overriding the `BaseModel.init()` method is the standard and better option to use. It requires the import of `tools` and is typically written as follows:

```
def init(self):
    tools.drop_view_if_exists(self.env.cr, self._table)
    self.env.cr.execute("""CREATE or REPLACE VIEW %s as (
                          SELECT
                          %s
                          FROM
                          %s
                          )""" % (self._table, self._select(), self._from()))
```

`tools.drop_view_if_exists` ensures that a conflicting view is not created when the SQL query is executed. It is standard to separate the different parts of the query to allow for easier model extension. Exactly how the query is split up across methods is not standardized, but at minimum, the `_select` and `_from` methods are common, and of course, all these methods will return strings.

See also

[Example: a SQL view using an override of `BaseModel.init\(\)`](#)

Use the model

Views and menu items for your SQL views are created and used in the same way as any other Odoo model. You are all set to start using your SQL view. Have fun!

A common mistake in SQL views is not considering the duplication of certain data due to table JOINS. This can lead to miscounting when using a field's `aggregator` and/or the pivot view. It is best to test your SQL view with sufficient data to ensure the resulting field values are as you expect.

Tip

If you have a field that you do not want as a measure (i.e., in your pivot or graph views), add `store=False` to it, and it will not show.

Get Help

[Contact Support](#)[Ask the Odoo Community](#)